

Password

Product Requirements Document (PRD)

Author: Soumya Bhattacharjee

1. Objective

Design, build, and launch an AI-Based Password Strength Estimator that:

- Evaluates the strength of a user-entered password using a combination of rule-based and ML-based analysis.
 - Clearly communicates a strength score and risk factors.
 - Generates actionable, easy-to-understand improvement suggestions and safer alternatives.
 - Can be embedded in consumer-facing experiences (web/app widget) and operated as a standalone tool.
-

2. Background / Context

Brief background

Most consumer applications currently provide simplistic password strength meters that rely on basic heuristics (length, character classes) and regex rules. These meters often:

- Overestimate strength for passwords that include predictable substitutions (e.g., "P@ssw0rd!").
- Do not account for modern cracking capabilities (GPU-driven brute force, dictionary attacks, password leaks).
- Give users minimal guidance on how to improve weak passwords.

Current state

- Users often reuse simple or guessable passwords across multiple accounts.
- Existing meters are non-standard: different sites give contradictory feedback for the same password.
- There is limited use of ML-based risk modeling that can:
 - Learn from leaked password corpora.
 - Estimate resistance to guessing attacks.
 - Identify semantic patterns (names, dates, keyboard patterns).

User pain

- Users do not know “how strong is strong enough” in a realistic threat model.
 - Users receive vague feedback (“Weak”, “Strong”) without understanding why.
 - Users lack concrete, personalized guidance on how to transform a weak password into a strong one.
 - As a result, they set weak passwords on high-value accounts and underestimate the risk.
-

3. Users and use cases

Primary personas

1. Everyday Internet User (“Rhea, 28, heavy social media user”)

- Goals:
 - Create passwords that keep social media, email, and banking accounts safe.
 - Avoid account takeovers and identity theft.
- Behaviors:
 - Uses similar passwords with small variations across sites.

- Relies on built-in password meters; rarely uses password managers.
- Constraints:
 - Limited security knowledge; overwhelmed by jargon.
 - Often on mobile; needs quick, clear feedback.

2. **Security-Conscious Power User (“Arjun, 32, tech-savvy professional”)**

- Goals:
 - Validate that chosen passwords meet high security standards.
 - Validate passphrases and password manager-generated secrets.
- Behaviors:
 - Uses password managers and 2FA, aware of breaches.
 - Experiments with long passphrases, wants a second opinion.
- Constraints:
 - Wants transparency on scoring logic and underlying assumptions.
 - Sensitive to data privacy (no storage of plaintext passwords).

3. **Developer / Integrator (“Maya, 30, full-stack engineer at a B2C app”)**

- Goals:
 - Embed a robust password strength estimator into signup and password reset flows.
 - Reduce account takeover incidents and support tickets.
- Behaviors:
 - Evaluates external components by ease of integration and UX quality.
- Constraints:
 - Limited time; prefers simple SDKs or APIs.
 - Must comply with security and privacy policies (no logging plaintext passwords).

Top use cases / jobs-to-be-done

1. **Quick strength check before using a password**
 - “When I am about to set a password for an important account, I want to quickly check how strong it is, so that I don’t unknowingly use a weak password for critical accounts.”
 2. **Guided improvement of an existing weak password**
 - “When the tool tells me my password is weak, I want clear, simple suggestions on how to improve it, so that I can fix it without needing deep technical knowledge.”
 3. **Assessment of passphrases and complex patterns**
 - “When I use a long phrase or unique pattern, I want to know if it is actually hard to guess, so that I can balance memorability and security.”
 4. **Developer integration into signup/reset forms**
 - “When I build sign-up and password-reset flows, I want to plug in a robust strength estimator, so that my users are nudged into creating stronger passwords and my app remains secure.”
 5. **Educational feedback to build better habits**
 - “When I test different passwords, I want to see why some are risky, so that I learn how to avoid common mistakes (e.g., personal info, common words).”
-

4. Scope

In scope (this iteration)

- **Core strength estimation engine**
 - Combine heuristic features (length, character classes, repetition, sequences) with an ML model trained on labeled password strength/risk data.
 - Generate a normalized strength score (e.g., 0–100) and categorical label (e.g., Very weak, Weak, Moderate, Strong, Very strong).
- **Suggestion generation**

- Generate human-readable suggestions on how to improve the current password:
 - Structural suggestions (length, complexity).
 - Pattern-based suggestions (avoid personal info, dictionary words).
- Generate 2–5 examples of stronger alternative passwords / passphrases based on the user's input pattern, without reusing sensitive substrings (names, emails).
- **Client-facing UX (reference implementation)**
 - A responsive web widget with:
 - Password input field with masking/unmask toggle.
 - Real-time strength meter (score + label + color).
 - Real-time list of suggestions.
 - Optional “Show example secure passwords” action.
- **Basic integration surface**
 - REST API contract for password evaluation (synchronous).
 - API key-based authentication for third-party integrators.
- **Privacy & security guardrails**
 - No persistent storage of raw passwords.
 - In-memory processing with optional hashed/logged aggregates (e.g., score distributions) only.
- **Basic analytics**
 - Capture anonymous metrics:
 - Distribution of strength scores.
 - Suggestion adoption proxy (e.g., user moving from weak to strong in a single session).

Out of scope (this iteration)

- Credential storage or password manager functionality.
 - Multi-factor authentication, login flows, or account lifecycle features.
 - Breach monitoring / credential stuffing detection based on username/email.
 - Cross-device sync of evaluated passwords or history.
 - Localization beyond English.
 - Native mobile apps; only a web-based reference implementation.
 - Enterprise admin dashboards or org-level reporting.
-

5. Functional requirements

Organized by priority: Must-have, Should-have, Nice-to-have.

5.1 Must-have

Epic: Core Strength Estimation Engine

- **Goal**
 - Provide an accurate, consistent, and explainable password strength score that reflects realistic cracking difficulty.
- **User story**
 - As an everyday internet user, I want a clear strength score for my password so that I know whether it is safe enough for my important accounts.
- **Detailed behavior**
 - **Inputs**
 - Required: `password` (string, length 1–256 characters).
 - Optional: `context` object (e.g., approximate intended use: “low-risk”, “high-risk”) for future tuning, but ignored in v1 if not needed.

- **Processing**

- Validate input:
 - Reject empty strings.
 - Reject strings longer than max allowed length (e.g., 256 chars).
 - Reject obviously non-password inputs (e.g., all whitespace).
- Extract features:
 - Length.
 - Character set usage (lowercase, uppercase, digits, symbols).
 - Repetition and sequences (e.g., “aaaaaa”, “123456”, “qwerty”).
 - Keyboard adjacency patterns where feasible (basic heuristics).
 - Presence of dictionary words (common English word list).
 - Leet speak and common substitutions (e.g., “P@ssw0rd”).
- Run ML model inference:
 - Model outputs a risk score or estimated “guess number” / crack time estimate, mapped to 0–100 normalized strength score.
- Derive categorical label:
 - Map score ranges to discrete labels and color codes.

- **Outputs**

- JSON object (for API) and UI binding (for widget):
 - **score**: integer 0–100.
 - **label**: enum [VERY_WEAK, WEAK, MODERATE, STRONG, VERY_STRONG].
 - **factors**: list of key factors contributing to the score (e.g., “Too short”, “Contains common word”, “Good length”, “Uses mixed

character sets”).

- **Validations and error states**
 - If `password` is missing: HTTP 400 with error code `MISSING_PASSWORD`.
 - If length > 256: HTTP 400 with error code `PASSWORD_TOO_LONG`.
 - If service is temporarily unavailable: HTTP 503 with `RETRY_AFTER` hint.

Epic: Suggestion Generation

- **Goal**
 - Provide clear, actionable guidance to help users transform weak passwords into strong ones.
- **User story**
 - As an everyday internet user, I want specific suggestions on how to improve my password so that I can quickly change it to something stronger.
- **Detailed behavior**
 - **Inputs**
 - `password` (string) and strength evaluation output from the core engine.
 - **Processing**
 - Derive suggestion rules from factors:
 - If length < N (configurable threshold, e.g., 12): suggest making it longer.
 - If lacking diversity (e.g., only letters): suggest adding digits and symbols.
 - If dictionary word detected: suggest using uncommon words or multi-word passphrase.

- If common patterns (sequences, repetitions): suggest breaking patterns.
- Generate suggestion list:
 - 2–5 concise suggestions, ordered by expected impact.
 - Language must be non-technical and user-friendly.
- Generate example passwords / passphrases:
 - Only when the user explicitly requests examples (e.g., clicks “Show examples”).
 - Use randomizable templates:
 - For weak passwords: transform structure (e.g., “WordWordDigitSymbol”).
 - Do not include personally identifiable substrings from the original input (e.g., parts that match common name/email/phone patterns).
- **Outputs**
 - **suggestions**: ordered list of strings (e.g., “Increase the length to at least 12 characters”).
 - **examples** (optional): 2–5 randomly generated strong password examples or passphrases.
- **Validations and error states**
 - If generation fails: return minimal fallback suggestion (“Use a long passphrase with multiple words and symbols.”) and log error without plaintext password.

Epic: Web Widget (Reference Implementation)

- **Goal**
 - Provide a simple, embeddable web component demonstrating real-time evaluation and suggestions.
- **User story**

- As a developer of a consumer app, I want a plug-and-play widget that shows strength feedback as users type their password so that I can improve password hygiene in my signup flow with minimal effort.
- **Detailed behavior**
 - **Inputs / interactions**
 - User types into a password field (masked by default).
 - Optional “eye” icon to toggle visibility (client-side only; no effect on backend).
 - **Processing**
 - On each keystroke (with debounce, e.g., 200–300 ms):
 - Call strength estimation API.
 - Update meter and suggestions in near real time.
 - **Outputs / UI elements**
 - Strength meter with:
 - Score and label.
 - Color bar (e.g., red → yellow → green).
 - List of suggestions.
 - Button/link “Show example strong passwords” to reveal examples.
 - **Validations and edge cases**
 - For very short passwords (e.g., <4 chars), show minimal message (“Too short to evaluate. Add more characters.”).
 - For empty field, hide meter and suggestions.
 - On API error:
 - Show non-blocking message (“Unable to evaluate right now. Try again.”).
 - Do not block user from continuing.

Epic: Privacy and Security Guardrails

- **Goal**
 - Ensure no plaintext password is stored and that data handling is privacy-preserving.
 - **User story**
 - As a security-conscious user, I want to know that my password is not stored or logged so that I can trust using this tool.
 - **Detailed behavior**
 - **Processing constraints**
 - No persistence of the raw password in databases or logs.
 - Logs may include:
 - Request timestamp.
 - Strength score.
 - High-level factor counts (e.g., length bucket, number of classes) but not actual characters.
 - Any debug logging during development must strip or mask passwords.
 - **Configuration**
 - Provide environment flag for extra logging in staging/debug builds that still never logs full plaintext.
-

5.2 Should-have

Epic: Configurable Policy Profiles

- **Goal**
 - Allow integrators to choose stricter or looser scoring profiles depending on risk level.

- **User story**
 - As a developer integrating this for a banking app, I want stricter thresholds so that passwords classified as “Strong” meet higher security requirements.
- **Detailed behavior**
 - **Inputs**
 - `policy_profile`: enum [DEFAULT, HIGH_SECURITY, LOW_SECURITY].
 - **Processing**
 - Adjust scoring thresholds and mapping for labels:
 - HIGH_SECURITY profile requires higher scores for “Strong” and “Very strong”.
 - **Outputs**
 - Same schema as core engine; labels reflect the applied profile.
 - **Edge cases**
 - If profile is missing or unsupported: default to `DEFAULT`.

Epic: Lightweight Explanation Layer

- **Goal**
 - Make results more transparent for advanced users without overwhelming novices.
- **User story**
 - As a power user, I want to understand why my password is considered strong or weak so that I can trust the scoring.
- **Detailed behavior**
 - **Outputs**
 - Optional `explanation` object containing:

- Key positive factors (e.g., “Length is 16 characters.”).
 - Key negative factors (e.g., “Contains common word ‘summer’.”).
 - UI toggles explanation on demand (e.g., “Why this score?” link).
-

5.3 Nice-to-have (future iterations)

Epic: Breach-Aware Risk Checks

- **Goal**
 - Incorporate whether a password appears in known breach datasets (via secure k-anonymity APIs, no raw password upload).
- **User story**
 - As a user, I want to know if my password has been seen in breaches so that I can avoid compromised passwords.
- **High-level behavior (future)**
 - Hash password client-side and use partial hash querying to external breach API.
 - Adjust risk score and show a warning if discovered in breach lists.

Epic: Multi-language Passphrase Guidance

- **Goal**
 - Provide localized suggestions and examples for non-English speakers.
 - **High-level behavior (future)**
 - Support language selection and localized dictionaries.
-

6. Acceptance Criteria

Functional correctness

- For a sample set of test passwords:
 - Common weak passwords (e.g., “password”, “123456”, “qwerty123”) consistently score in the “Very weak” or “Weak” range.
 - Reasonable complex passwords (e.g., 12+ characters with mixed character types and no common words) score “Strong” or “Very strong”.
 - Long but simple phrases (e.g., multi-word passphrases) score at least “Moderate” and higher when uncommon words are used.

UX and behavior

- Real-time feedback:
 - Strength meter and suggestions update within 500 ms for 95% of keystrokes in a typical network environment.
- Clarity:
 - At least 80% of users in qualitative tests can:
 - Explain why a password is considered weak or strong based on the UI.
 - Describe at least one concrete step to improve a weak password.
- Suggestion adoption (proxy metric post-launch):
 - In at least 30% of sessions where the initial password is “Weak” or “Very weak”, the user modifies the password and reaches “Moderate” or higher within the same session.

Performance

- API latency:

- P95 end-to-end API response time ≤ 300 ms for password length up to 64 characters under normal load.
- Availability:
 - Target 99.5% uptime for the evaluation API in production (excluding planned maintenance).
- Scalability:
 - System sustains at least X requests per second (to be defined with infra) with linear horizontal scaling.

Security and privacy

- No plaintext passwords are stored in any persistent storage or logs (validated by code review and log sampling).
- Penetration test and security review completed with no critical issues outstanding for launch.
- Inputs are validated against injection and abuse patterns (e.g., extremely long inputs, attempts to inject code in fields).

Integration

- API documentation:
 - Clearly specifies endpoints, request/response schemas, error codes, and rate limits.
 - Reference widget can be integrated into a demo page in under 2 hours by a new engineer following documentation.